



VAPT TASK - 1

A) Theoretical Knowledge

1. Understanding Security Assessment

A **Security Assessment Framework** is a structured methodology using standards, guidelines, and best practices (like **NIST CSF**, **OWASP ASVS**) to help organizations systematically **Identify, Protect, Detect, Respond, and Recover** from cyber risks, enabling a holistic view of security posture, gap analysis, and continuous improvement through maturity levels (e.g., non-compliant to fully compliant). These frameworks guide audits, testing, and monitoring to ensure controls meet objectives, often leveraging tools and integrating with existing processes for consistent risk management.

Key Components & Steps

- **Define Scope & Objectives:** Understand what's being assessed (network, app, cloud) and the goals (compliance, risk reduction).
- **Establish Baseline:** Compare current security state against a chosen standard (e.g., NIST, OWASP).
- **Conduct Analysis:** Perform gap analysis, vulnerability scanning, penetration testing, and code review.
- **Maturity Assessment:** Evaluate the effectiveness of controls (e.g., Fully-Compliant, Partially-Compliant, Non-Compliant).
- **Report & Remediate:** Generate clear reports with findings and actionable workarounds, then prioritize fixes.
- **Continuous Improvement:** Implement continuous monitoring and regular re-assessments for ongoing security posture improvement.

Common Frameworks & Standards

- **NIST Cybersecurity Framework (CSF):** Focuses on Govern, Identify, Protect, Detect, Respond, Recover functions.
- **OWASP Application Security Verification Standard (ASVS):** Specific for web/mobile app security, driving standardized testing.
- **Cloud App Security Assessment (CASA):** Tiered, risk-based approach for cloud apps, often using OWASP ASVS.
- **Data Security Council of India (DSCI) Assessment Framework (DAF):** Provides maturity metrics for various security disciplines.

Why use Frameworks?

Frameworks are used to:

- Avoid missed vulnerabilities
- Standardize testing
- Gain client trust
- Meet compliance
- Improve reports
- Protect you legally



The NIST CSF 2.0 Framework

The **NIST Cybersecurity Framework (CSF) 2.0** is a globally recognized framework developed by the **National Institute of Standards and Technology (NIST)** to help organizations **manage and reduce cybersecurity risk**. CSF 2.0 provides a **structured, risk-based approach** for identifying, protecting, detecting, responding to, and recovering from cybersecurity threats, while emphasizing governance and accountability.

The primary objective of NIST CSF 2.0 is to:

- Establish a **common cybersecurity language** across technical, management, and executive teams
- Enable organizations to **assess current cybersecurity posture**
- Identify gaps and prioritize risk mitigation activities
- Align cybersecurity practices with **business objectives and regulatory requirements**

Core Functions of NIST CSF 2.0

NIST CSF 2.0 is structured around six core functions, which represent the lifecycle of cybersecurity risk management.

Govern

This function emphasizes **leadership, strategy, and oversight**. It encourages senior management to integrate cybersecurity into enterprise risk management, establish policies, and ensure accountability for risk decisions across the organization.

Identify

Focuses on understanding the organization's environment, assets, data, systems, and potential risks. This helps organizations know what needs protection and the possible impact of cyber threats.

Protect

Involves implementing safeguards to prevent or limit the impact of cybersecurity incidents. Measures include access controls, employee training, data protection, and secure system maintenance.



Figure 1: NIST CSF 2.0 Framework



Detect

Supports timely identification of cybersecurity events through continuous monitoring, anomaly detection, and security logging.

Respond

Covers actions taken after detecting an incident, including incident response planning, communication, mitigation, and corrective actions to reduce damage.

Recover

Ensures that systems and operations affected by a cybersecurity incident are restored. Recovery planning, system restoration, and communication with stakeholders help organizations return to normal operations and improve resilience.

Types of Security Testing

Security testing is the process of evaluating systems, applications, or networks to identify vulnerabilities, threats, and weaknesses. Its main goal is to ensure the **confidentiality, integrity, and availability** of information. Different types of security testing focus on various aspects of security and risk management.

1. Vulnerability Scanning

It is a type of testing that uses automated tools to scan the system for known vulnerabilities and weaknesses. It aims to detect patterns of vulnerabilities that are commonly exploited by attackers.

2. Security Scanning

It involves identifying weaknesses in the network or system and then providing solutions to mitigate these risks. It can be performed either manually or automatically, depending on the complexity of the system.

3. The main difference between vulnerability scanning and security scanning is that vulnerability scanning focuses specifically on known vulnerabilities that could be exploited by attackers whereas security scanning evaluates overall security posture, including vulnerabilities, configurations, and potential risks.

4. Penetration Testing

Penetration Testing simulates an attack from a malicious hacker to identify vulnerabilities in the system. This type of testing helps organizations understand how an attacker might exploit weaknesses in the system.

5. Risk Assessment

It involves analyzing the security risks that could affect the organization. Risks are categorized as low, medium, or high, and this testing suggests controls and measures to minimize those risks.

6. Security Auditing

internal inspection of the system to identify security defects. This can involve reviewing system configurations, checking for weaknesses in the code, or conducting a line-by-line inspection of the application's source code. Security audits ensure that all security standards and protocols are being followed and identify any gaps that need to be addressed.

Other Types of Security Testing

- API Security Testing
- Mobile application security
- Network Security Testing



2. VAPT Methodology

Vulnerability Assessment and Penetration Testing (VAPT) is a structured and systematic methodology used to evaluate the security posture of information systems, applications, and networks. It combines the identification of security weaknesses with controlled exploitation to assess the potential impact of vulnerabilities in real-world scenarios. The primary objective of VAPT is to proactively identify and mitigate security risks before they can be exploited by malicious actors.

Phases of the VAPT Methodology

1. Planning and Scope Definition

This phase establishes the objectives, scope, and boundaries of the assessment. It defines the systems to be tested, testing approach (black-box, white-box, or gray-box), and rules of engagement. Proper authorization and alignment with business and compliance requirements are ensured during this phase.

2. Information Gathering (Reconnaissance)

In this phase, relevant information about the target environment is collected to understand the attack surface. This may include identifying network ranges, domains, services, technologies, and system architecture. Effective reconnaissance helps in identifying potential attack vectors.

3. Vulnerability Assessment

Automated tools and manual techniques are used to identify known vulnerabilities, security misconfigurations, and weaknesses within the target systems. Identified vulnerabilities are analyzed and categorized based on their severity and potential impact. This phase focuses on vulnerability identification rather than exploitation.

4. Penetration Testing

Penetration testing involves controlled attempts to exploit identified vulnerabilities to determine their exploitability and real-world impact. This phase validates whether vulnerabilities can lead to unauthorized access, privilege escalation, or data compromise.

5. Post-Exploitation Analysis

Once exploitation is successful, this phase assesses the extent of access obtained and evaluates potential damage. It examines lateral movement, data exposure, and the overall impact on the organization's assets to understand the severity of the compromise.

6. Reporting and Documentation

A detailed report is prepared summarizing the findings, exploited vulnerabilities, associated risks, and business impact. The report includes prioritized remediation recommendations and mitigation strategies tailored for both technical teams and management.



7. Remediation and Re-Testing

Following remediation efforts by the organization, re-testing is conducted to verify that identified vulnerabilities have been effectively resolved. This phase ensures continuous improvement and strengthens the overall security posture.

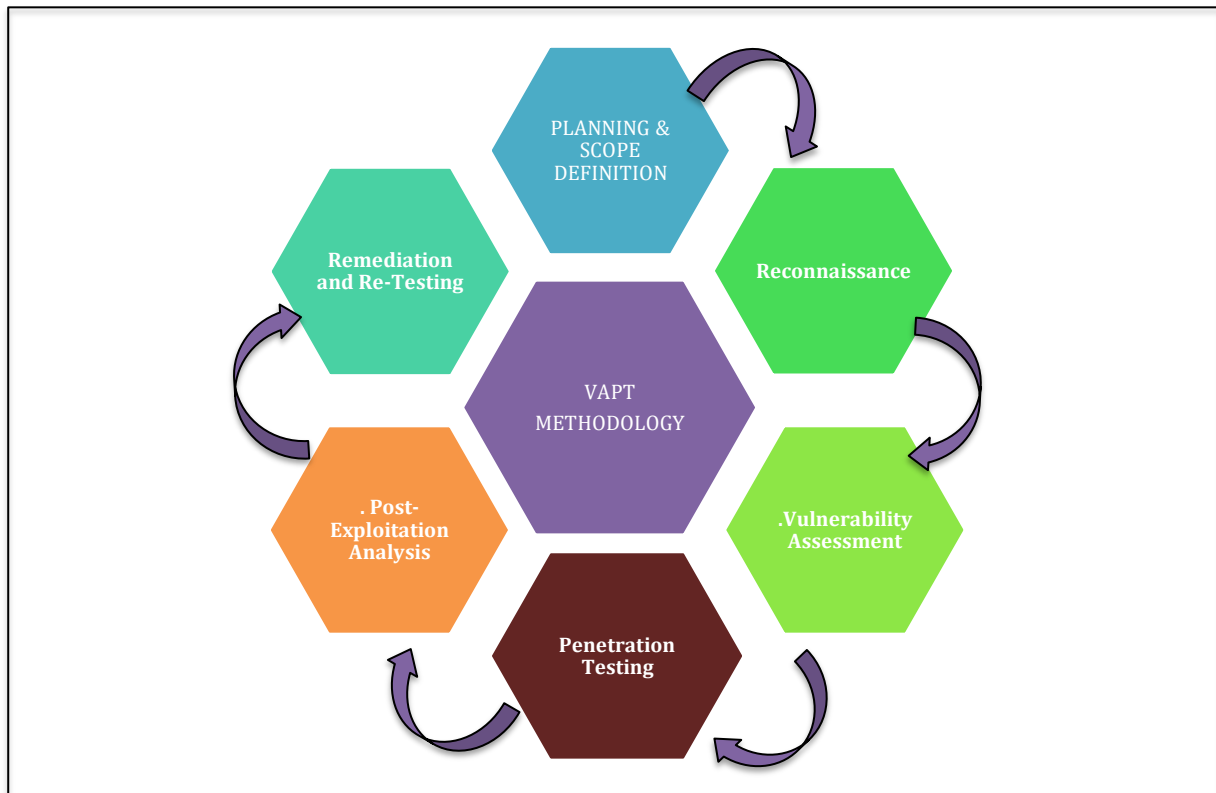


Figure 2:VAPT Methodology

OWASP Web Security Testing Framework (WSTG) ^[6]

The **OWASP Web Security Testing Framework (WSTG)** is a comprehensive and structured guide developed by the **Open Web Application Security Project (OWASP)** to help organizations test the security of web applications. It provides detailed methodologies, techniques, and best practices for identifying security vulnerabilities throughout the web application lifecycle. The framework is widely used by security professionals, developers, and testers as a standard reference for web application security testing.

Purpose of OWASP WSTG

- Provide a standardized approach for web application security testing
- Help identify vulnerabilities before they are exploited by attackers
- Improve the overall security posture of web applications
- Support secure development and compliance requirements



Structure of OWASP WSTG ^[6]

The Five Phases of the OWASP Web Security Testing Framework

- 1) *Before Development Begins*: This initial phase involves establishing the overall security policy for the application and the organization. Key activities include defining security requirements and identifying the scope of testing.
- 2) *During Definition and Design*: In this phase, security considerations are integrated into the application's design. This typically involves performing threat modeling to identify potential attack vectors and designing security controls to mitigate identified risks.
- 3) *During Development*: This phase focuses on the actual implementation of security measures during coding. Activities include source code reviews to find vulnerabilities and ensuring that secure coding practices are followed.
- 4) *During Deployment*: Security testing during deployment involves ensuring the application and its infrastructure are securely configured before going live. This includes configuration and deployment management testing, as well as final penetration testing to validate the security posture in the production environment.
- 5) *During Maintenance and Operations*: This final phase emphasizes continuous security monitoring and assessment after the application is operational. It involves ongoing testing, incident response planning, and regular updates to maintain the application's security over time.

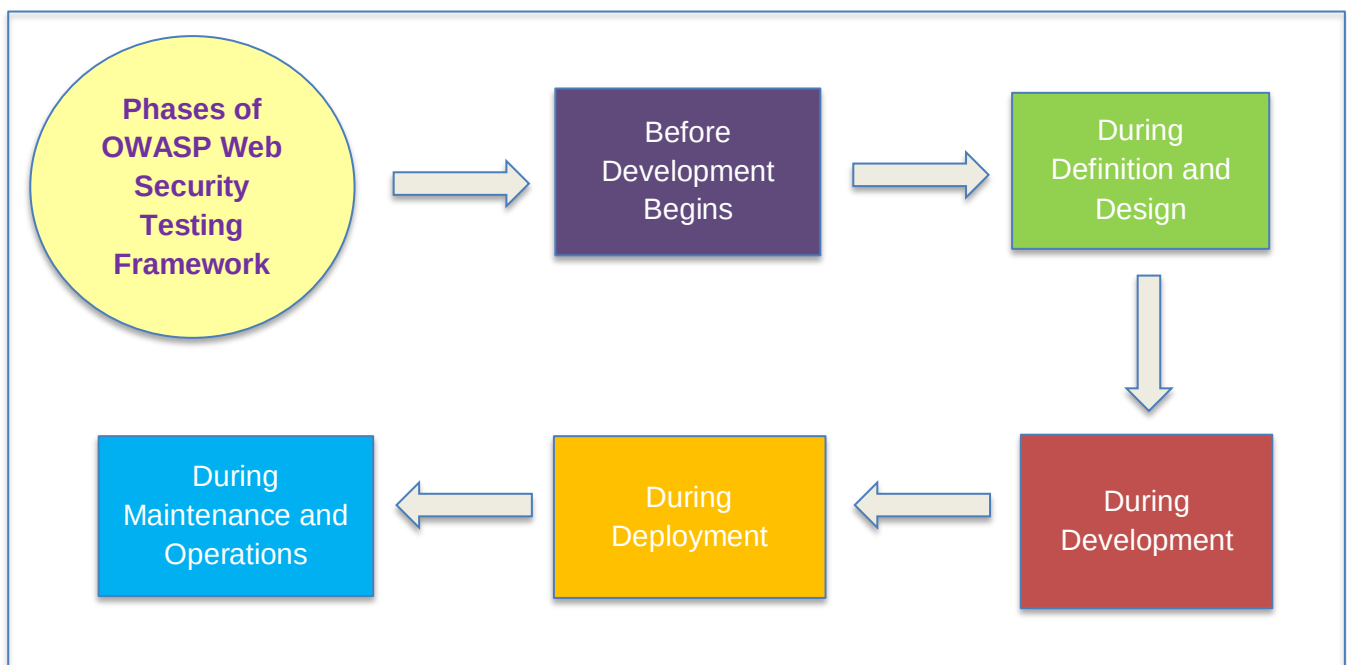


Figure 3: Phases of OWASP WSTG

Benefits of OWASP WSTG

- Provides a **detailed and standardized testing methodology**
- Covers both **technical vulnerabilities and logical flaws**
- Widely accepted and continuously updated by the security community
- Supports integration with **VAPT, Secure SDLC, and compliance programs**



3. Security Standards & Compliance ^[7]

Security standards and compliance refer to established frameworks, guidelines, and regulations that organizations follow to protect information systems, data, and digital assets. These standards define best practices for implementing security controls, managing risks, and ensuring the confidentiality, integrity, and availability of information, whereas Compliance ensures that an organization meets legal, regulatory, and industry requirements, reducing the risk of data breaches, legal penalties, and reputational damage.

Common Security Standards and Frameworks

- ISO/IEC 27001
- NIST Cybersecurity Framework (CSF)
- PCI DSS
- HIPAA
- GDPR
- COBIT
- ITIL

ISO/IEC 27001 is an international standard that specifies requirements for establishing, implementing, maintaining, and continuously improving an *Information Security Management System (ISMS)*. It focuses on identifying information security risks and applying appropriate controls to mitigate them. It emphasizes risk assessment and risk treatment.

The *NIST Cybersecurity Framework* is a risk-based framework developed to help organizations manage and reduce cybersecurity risks.

Payment Card Industry Data Security Standard (PCI DSS) is a mandatory security standard for organizations that store, process, or transmit credit and debit card information. It aims to protect cardholder data and reduce payment fraud.



Figure 4: Security standards and compliance



Health Insurance Portability and Accountability Act (HIPAA) is a U.S. regulation that establishes security and privacy requirements for protecting electronic Protected Health Information (ePHI).

General Data Protection Regulation (GDPR) is a comprehensive data protection regulation enforced in the European Union, governing the processing of personal data.

Control Objectives for Information and Related Technologies framework (COBIT) focuses on IT governance and management, including information security.\

The *Information Technology Infrastructure Library (ITIL)* is a framework of best practices for IT service management across the service lifecycle. ITIL provides a collection of best practices regarding IT management. These practices include recommendations about embedding security compliance at every lifecycle stage.

OWASP Top 10 ^[8]

The Ten Most Critical Web Application Security Risks

- **A01:2025 - Broken Access Control** maintains its position at #1 as the most serious application security risk; the contributed data indicates that on average, 3.73% of applications tested had one or more of the 40 Common Weakness Enumerations (CWEs) in this category. As indicated by the dashed line in the above figure, Server-Side Request Forgery (SSRF) has been rolled into this category.
- **A02:2025 - Security Misconfiguration** moved up from #5 in 2021 to #2 in 2025. Misconfigurations are more prevalent in the data for this cycle. 3.00% of the applications tested had one or more of the 16 CWEs in this category. This is not surprising, as software engineering is continuing to increase the amount of an application's behaviour that is based on configurations.
- **A03:2025 - Software Supply Chain Failures** is an expansion of A06:2021-Vulnerable and Outdated Components to include a broader scope of compromises occurring within or across the entire ecosystem of software dependencies, build systems, and distribution infrastructure. This category was overwhelmingly voted a top concern in the community survey. This category has 5 CWEs and a limited presence in the collected data, but we believe this is due to challenges in testing and hope that testing catches up in this area. This category has the fewest occurrences in the data, but also the highest average exploit and impact scores from CVEs.
- **A04:2025 - Cryptographic Failures** falls two spots from #2 to #4 in the ranking. The contributed data indicates that, on average, 3.80% of applications have one or more of the 32 CWEs in this category. This category often leads to sensitive data exposure or system compromise.
- **A05:2025 - Injection** falls two spots from #3 to #5 in the ranking, maintaining its position relative to Cryptographic Failures and Insecure Design. Injection is one of the most tested categories, with the greatest number of CVEs associated with the 38 CWEs in this category. Injection includes a range of issues from Cross-site Scripting (high frequency/low impact) to SQL Injection (low frequency/high impact) vulnerabilities.



- **A06:2025 - Insecure Design** slides two spots from #4 to #6 in the ranking as Security Misconfiguration and Software Supply Chain Failures leapfrog it. This category was introduced in 2021, and we have seen noticeable improvements in the industry related to threat modeling and a greater emphasis on secure design.
- **A07:2025 - Authentication Failures** maintains its position at #7 with a slight name change (previously it was "Identification and Authentication Failures") to more accurately reflect the 36 CWEs in this category. This category remains important, but the increased use of standardized frameworks for authentication appears to be having beneficial effects on the occurrences of authentication failures.
- **A08:2025 - Software or Data Integrity Failures** continues at #8 in the list. This category is focused on the failure to maintain trust boundaries and verify the integrity of software, code, and data artifacts at a lower level than Software Supply Chain Failures.
- **A09:2025 - Security Logging & Alerting Failures** retains its position at #9. This category has a slight name change (previously Security Logging and Monitoring Failures") to emphasize the importance of the alerting functionality needed to induce appropriate action on relevant logging events. Great logging with no alerting is of minimal value in identifying security incidents. This category will always be underrepresented in the data, and was again voted into a position in the list from the community survey participants.
- **A10:2025 - Mishandling of Exceptional Conditions** is a new category for 2025. This category contains 24 CWEs focusing on improper error handling, logical errors, failing open, and other related scenarios stemming from abnormal conditions that systems may encounter.

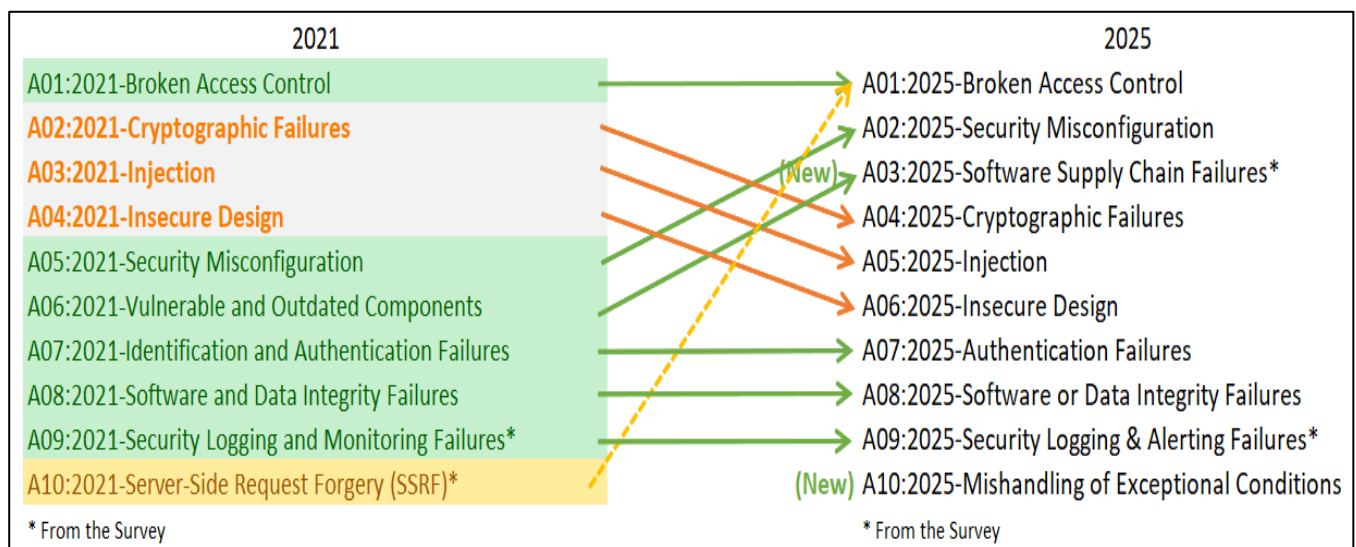


Figure 3:OWASP Top 10 2025



4. Risk Assessment Basics

CVSS Calculator ^[13]

The **CVSS Calculator** is a standardized method used to assess the **severity of security vulnerabilities**. It assigns a **numerical score between 0.0 and 10.0**, indicating how serious a vulnerability is. CVSS is widely used in vulnerability management, VAPT reports, and compliance audits.

CVSS scoring assigns a number from 0 to 10 based on three main factors: Base, Temporal, and Environmental metrics. The Base score shows the inherent characteristics of a vulnerability. The Temporal score considers how those characteristics may change over time.

CVSS Score	Qualitative Rating
0.0	None
0.1 – 3.9	Low
4.0 – 6.9	Medium
7.0 – 8.9	High
9.0 – 10.0	Critical

How is a CVSS score calculated?

A CVSS score is based on three sets of metrics: Base, Temporal, and Environmental. Each of these has its own scoring elements.

Source 1: balbix.com:

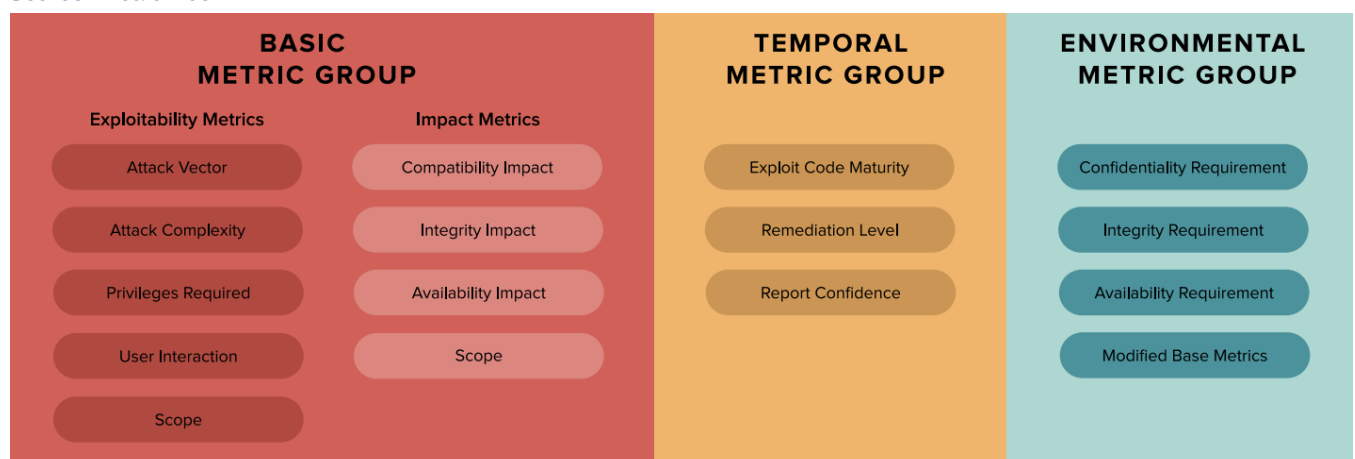


Figure 4: CVSS Score Metrics



CVSS Base Metrics

The Base Metrics are the core components used to determine how severe a security vulnerability is. They focus on the vulnerability's characteristics, regardless of whether it has been exploited or mitigated. These metrics include Exploitability, Scope, and Impact.

Exploitability: This metric assesses how easily a vulnerability is exploited. It is broken down into four sub-components:

- **Attack Vector:** Measures how an attack can be executed, with higher scores for remote attacks versus those requiring physical access.
- **Attack Complexity:** Evaluate the difficulty of executing the attack, with lower scores for easier vulnerabilities to exploit.
- **Privileges Required:** This indicator indicates the level of access needed to exploit the vulnerability, with higher scores for attacks requiring fewer privileges.
- **User Interaction:** Considers whether the attacker needs to involve a user in the exploit, with autonomous attacks scoring higher.

Scope: This metric assesses whether the vulnerability can affect other components beyond the initial target. The score will be higher if the vulnerability can propagate, such as compromising an entire system through a single application flaw.

Impact: This metric evaluates the potential consequences of a successful exploit, focusing on three areas:

- **Confidentiality:** Measures the extent of data exposure.
- **Integrity:** Assesses the ability of the attacker to modify data.
- **Availability:** Evaluate the potential disruption to system access and functionality.

CVSS Temporal Metrics

CVSS Temporal Metrics evaluate the changing nature of a vulnerability over time. These metrics assess a vulnerability's current exploitability and the availability of remediating controls, such as patches. Key subcomponents of Temporal Metrics include:

- **Exploit Code Maturity:** A vulnerability is less threatening until a method to exploit it becomes available. As exploit code matures and becomes more widespread, its associated score increases, reflecting the heightened risk.
- **Remediation Level:** A vulnerability may not initially have a patch or workaround. As temporary fixes or official patches are released, the vulnerability score decreases, indicating reduced risk.
- **Report Confidence:** This measures how well a vulnerability is validated, ensuring it is both real and exploitable—higher confidence results in a higher score.

CVSS Environmental Metrics

CVSS Environmental Metrics allow organizations to adjust the Base CVSS score based on their specific Security Requirements and modifications of Base Metrics.

- **Security Requirements:** These metrics consider the criticality of the affected asset. Mission-critical systems, like a database containing all customer data, receive higher scores than less critical assets, such as a non-privileged user's workstation.
- **Modified Base Metrics:** Organizations can modify Base CVSS Metrics based on existing mitigations. For instance, "air gapping" a server—disconnecting it from external networks—lowers the Attack Vector score since remote exploitation is no longer possible.



By considering both Temporal and Environmental Metrics, organizations can achieve a more tailored and accurate assessment of a vulnerability's actual risk to their specific environment.

Risk Matrix

A **Risk Matrix** is a visual and analytical tool used in risk assessment to **evaluate and prioritize risks** by considering two key factors: **likelihood** (probability of occurrence) and **impact** (severity of consequences). It helps organizations categorize risks as **Low, Medium, or High**, enabling effective decision-making and resource allocation.

The purpose of a risk matrix is straightforward: to provide a clear, visual representation of risks that facilitates quicker and more accurate decision-making. By mapping risks onto a matrix, organizations can readily see which risks require immediate attention and which can be monitored over time.

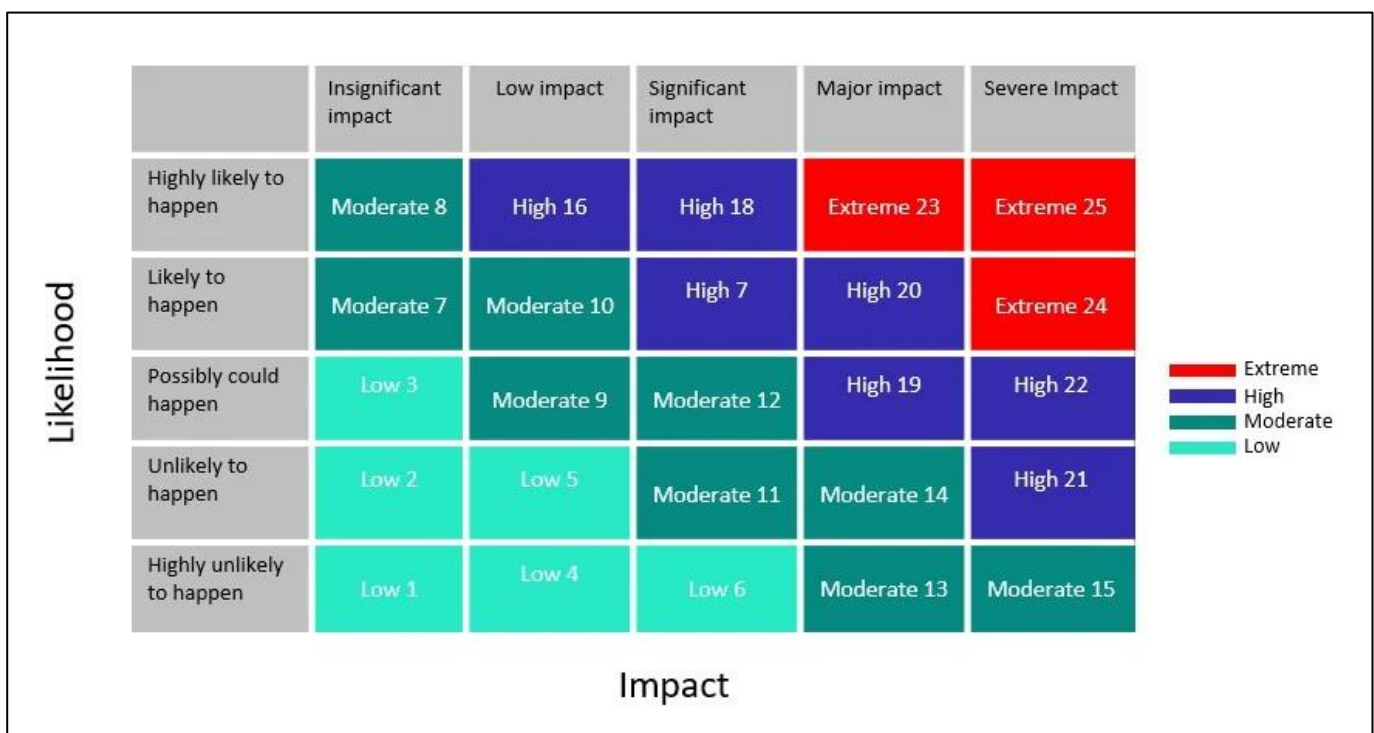


Figure 5: Risk Matrix

Severity of Risk according to CVSS:

- **None:** 0.0 (No severity)
- **Low:** 0.1 - 3.9 (Minor risk)
- **Medium:** 4.0 - 6.9 (Moderate risk)
- **High:** 7.0 - 8.9 (Serious risk, needs immediate attention)
- **Critical:** 9.0 - 10.0 (Most urgent, requires immediate action)



5. Common Vulnerabilities

Network vulnerabilities are weaknesses or flaws in network infrastructure, configurations, protocols, or security controls that can be exploited by attackers to gain unauthorized access, disrupt services, or compromise data. These vulnerabilities affect devices such as routers, switches, firewalls, servers, and communication links, and they pose significant risks to an organization's overall security posture. ^[14]

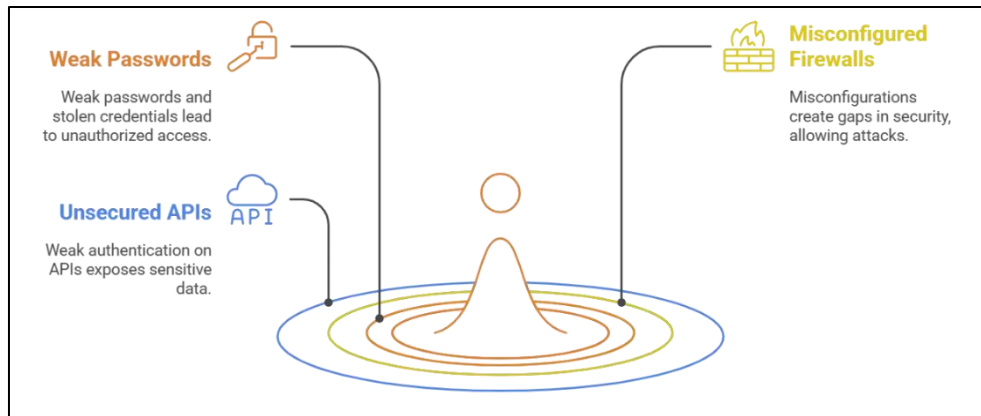


Figure 6: Network Vulnerabilities

Common Types of Network Vulnerabilities

1. Misconfigured Network Devices

Improper configurations of routers, switches, firewalls, and load balancers can expose the network to unauthorized access.

Examples:

- Default credentials on network devices
- Open or unnecessary ports and services
- Improper firewall or ACL rules

2. Unpatched or Outdated Systems

Network devices and servers running outdated firmware or operating systems may contain known vulnerabilities that attackers can exploit.

Impact:

- Remote code execution
- Privilege escalation

3. Weak Authentication and Access Control

Inadequate authentication mechanisms can allow unauthorized users to access network resources.

Examples:

- Weak or shared passwords
- Lack of multi-factor authentication (MFA)
- Excessive user privileges

4. Insecure Network Protocols

Use of legacy or insecure protocols can expose sensitive data to interception or manipulation.

Examples:

- Telnet, FTP, HTTP
- Weak encryption in older protocols
- Unsecured SNMP configurations



5. Lack of Network Segmentation

Flat network architectures allow attackers to move laterally once they gain initial access.

Risks:

- Rapid spread of malware

- Unauthorized access to critical systems

- Increased impact of breaches

6. Denial of Service (DoS) Exposure

Networks that lack rate limiting or protection mechanisms are vulnerable to DoS and DDoS attacks.

Impact:

- Service outages

- Degraded network performance

- Business disruption

7. Insecure Wireless Networks

Wireless networks that are poorly secured can provide attackers with easy entry points.

Examples:

- Weak Wi-Fi encryption (WEP, weak WPA settings)

- Open or rogue access points

- Lack of network isolation for guest Wi-Fi

Web vulnerabilities are weaknesses in web applications, web services, or their underlying components that can be exploited by attackers to compromise data, disrupt services, or gain unauthorized access. These vulnerabilities typically arise from insecure coding practices, poor input validation, misconfigurations, and weak authentication or authorization mechanisms.

Common Types of Web Vulnerabilities

1. Injection Attacks

Injection vulnerabilities occur when untrusted user input is improperly processed by the application.

Examples:

- SQL Injection

- Command Injection

- LDAP Injection

Impact: Data leakage, unauthorized access, database manipulation.

2. Cross-Site Scripting (XSS)

XSS vulnerabilities allow attackers to inject malicious scripts into web pages viewed by other users.

Types:

- Stored XSS

- Reflected XSS

- DOM-based XSS

Impact: Session hijacking, credential theft, defacement.

3. Broken Authentication

Occurs when authentication mechanisms are improperly implemented.

Examples:

- Weak password policies

- Insecure session handling

- Lack of multi-factor authentication

Impact: Account takeover and unauthorized access.



4. Broken Access Control

Allows users to perform actions or access resources beyond their authorized privileges.

Examples:

Insecure Direct Object References (IDOR)

Privilege escalation

Impact: Data exposure and unauthorized system operations.

[12]

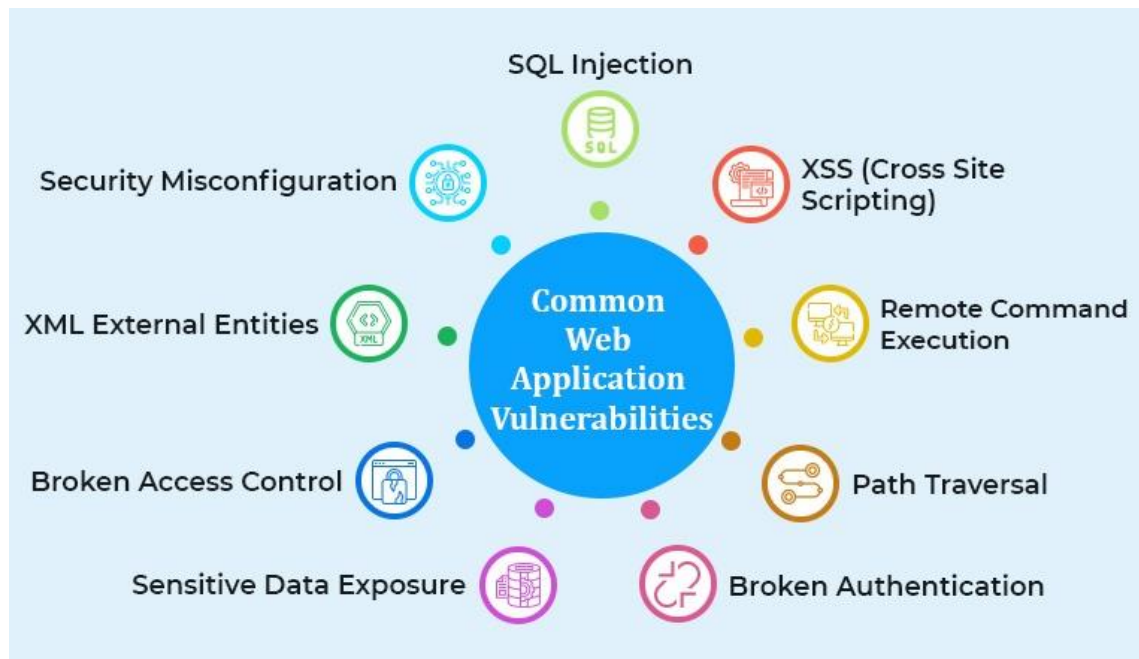


Figure 7: Web Vulnerabilities

5. Security Misconfiguration

Results from insecure default settings or incomplete configurations.

Examples:

Default credentials

Open admin interfaces

Missing security headers

Impact: Increased attack surface and system compromise.

6. Cross-Site Request Forgery (CSRF)

Forces authenticated users to perform unwanted actions without their knowledge.

Impact: Unauthorized transactions or account changes.

7. Sensitive Data Exposure / Cryptographic Failures

Occurs when sensitive data is not properly protected.

Examples:

Weak encryption

Plaintext data transmission

Poor key management

Impact: Data breaches and privacy violations.

8. Server-Side Request Forgery (SSRF)

Allows attackers to make the server initiate unintended network requests.

Impact: Internal network access and data exposure.



B) Practical Application

1. Setup Testing Environment

Install Kali Linux and Metasploitable on VirtualBox.

Connect both the VMs to each other by creating a Network Interface.

2. Vulnerability Scanning with Nikto Scanning tool

Scanning metasploitable with Kali Linux using Nikto.

```
(root@kali) ~  
# nikto -h 192.168.64.3  
- Nikto v2.5.0  
  
+ Target IP: 192.168.64.3  
+ Target Hostname: 192.168.64.3  
+ Target Port: 80  
+ Start Time: 2026-01-01 16:20:11 (GMT5.5)  
  
+ Server: Apache/2.2.8 (Ubuntu) DAV/2  
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.  
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options  
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/  
+ /index: Uncommon header 'tcn' found, with contents: list.  
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275  
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.  
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.  
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing  
+ /phpinfo.php: Output from the phpinfo() function was found.  
+ /doc/: Directory indexing found.  
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678  
+ /3=PHPE8B85F2A0-3C92-11d3-A3A0-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184  
+ /3=PHPE9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184  
+ /3=PHPE9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184  
+ /3=PHPE9568F35-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184  
+ /phpMyAdmin/ChangeLog.php: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.  
+ /phpMyAdmin/ChangeLog: Server may leak inodes via ETags, header found with file /phpMyAdmin/ChangeLog, inode: 92462, size: 40540, mtime: Tue Dec 9 22:54:00 2008. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418  
+ /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.  
+ /test/: Directory indexing found.  
+ /test/: This might be interesting.  
+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. See: CWE-552  
+ /icons/: Directory indexing found.  
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/  
+ /phpMyAdmin/: phpMyAdmin directory found.  
+ /phpMyAdmin/Documentation.html: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.  
+ /phpMyAdmin/README: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts. See: https://typo3.org/  
+ /wp-config.php#: wp-config.php file found. This file contains the credentials.  
+ 8910 requests: 0 error(s) and 27 item(s) reported on remote host  
+ End Time: 2026-01-01 16:22:04 (GMT5.5) (113 seconds)  
  
+ 1 host(s) tested
```

The Findings:

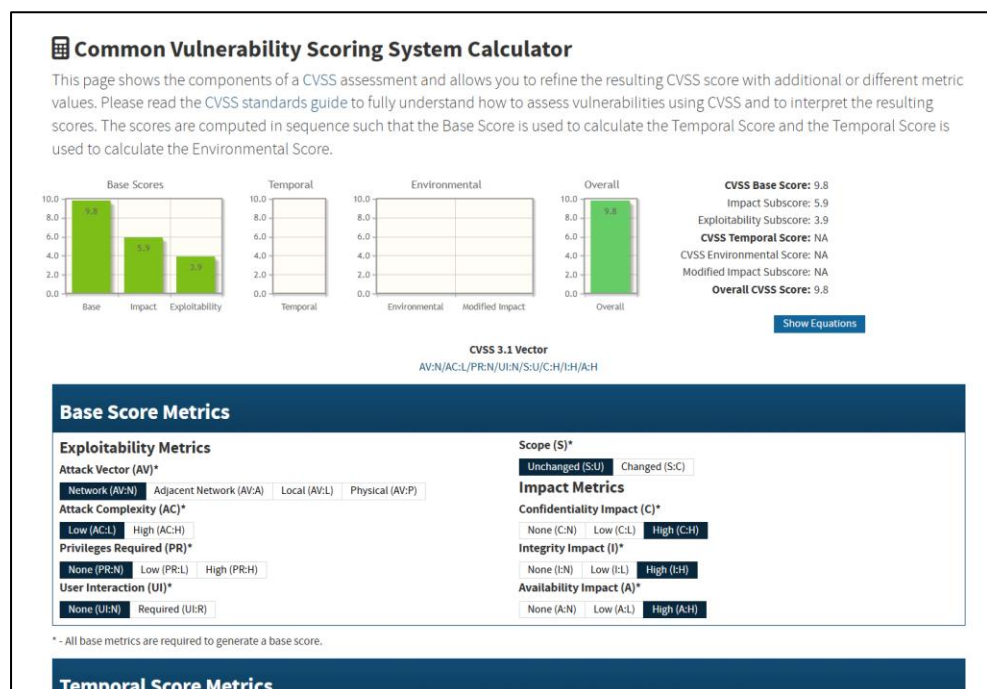


Figure 8: CVSS-NDV Calculator



Vulnerability Tracking				
Finding	CVE / ID	CVSS 3.1 Score	Severity	Description
Exposed wp-config.php	N/A	9.8	CRITICAL	The configuration file for WordPress was found. This contains plaintext database credentials and secret keys.
Outdated Apache (2.2.8)	CVE-2008-0455+	7.5	HIGH	The server runs an EOL version of Apache with multiple known vulnerabilities including memory leaks and DoS.
HTTP TRACE Enabled	CVE-2003-1418	5.3	MEDIUM	Vulnerable to Cross-Site Tracing (XST), allowing attackers to steal session cookies via HTTP headers.
PHP info() Exposed	CWE-552	5.3	MEDIUM	The phpinfo() page leaks environment variables, loaded modules, and internal file paths.
Directory Indexing	CVE-1999-0678	5	MEDIUM	Directories like /doc/ and /test/ are browseable, allowing attackers to map the web structure.
Missing Security Headers	N/A	4.3	LOW	Missing X-Frame-Options and X-Content-Type-Options increases risk of Clickjacking and MIME sniffing.
*Reference: https://youtu.be/Q6B2wjQdRxY?si=aGOBN944G2hMWMK6				

Risk Matrix			
Likelihood \ Impact	Low	Medium	High
Low			phpinfo() Leak
Medium		HTTP TRACE	Outdated Apache
High	Missing Headers	Directory Indexing	wp-config.php Exposure
*References: 1) https://owasp.org/www-community/OWASP_Risk_Rating_Methodology			



References

- [1] <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- [2] <https://www.geeksforgeeks.org/software-testing/security-testing/>
- [3] <https://www.sisainfosec.com/blogs/10-types-of-security-testing-techniques/>
- [4] <https://qualysec.com/what-is-vapt-testing-its-methodology-importance-for-business/>
- [5] <https://wesecureapp.com/blog/understanding-vapt-simple-guide-to-vulnerability-assessment-and-penetration-testing/>
- [6] https://owasp.org/www-project-web-security-testing-guide/latest/3-The_OWASP_Testing_Framework/0-The_Web_Security_Testing_Framework
- [7] <https://owasp.org/www-project-top-ten/>
- [8] <https://owasp.org/Top10/2025/>
- [9] <https://www.balbix.com/insights/understanding-cvss-scores/>
- [10] <https://www.metricstream.com/learn/what-is-risk-matrix.html>
- [11] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology
- [12] <https://www.sayonetech.com/blog/web-application-security/>
- [13] www.chatgpt.com
- [14] www.google.com